

30.yolov26模型部署(从0到1版)

1. windows搭建环境

1.1 安装miniconda

miniconda安装

- 下载地址: <https://mirrors.tuna.tsinghua.edu.cn/anaconda/miniconda/>
- 安装在非C盘, 节省C盘空间, 且安装目录中不要有中文
- 勾选将其添加到PATH, 添加到环境变量

conda环境创建

代码块

```
1 conda create -n yolov26 python=3.8
```

明确指定版本, 否则可能会因版本过高导致有包装不上

pypi配置国内源

清华源:<https://mirrors.tuna.tsinghua.edu.cn/help/pypi/>

代码块

```
1 pip config set global.index-url  
https://mirrors.tuna.tsinghua.edu.cn/pypi/web/simple
```

1.2 安装pytorch

pytorch安装

官方地址:<https://pytorch.org/>

在一个单独的环境中, 能使用pip就尽量使用pip, 实在有问题的情况, 例如没有合适的编译好的系统版本的安装包, 再使用conda进行安装, 不要来回混淆CUDA

代码块

```
1 conda install pytorch==1.12.1 torchvision==0.13.1 torchaudio==0.12.1  
cudatoolkit=11.3 -c pytorch
```

1.3 安装ultralytics

- 方法一：pip直接安装

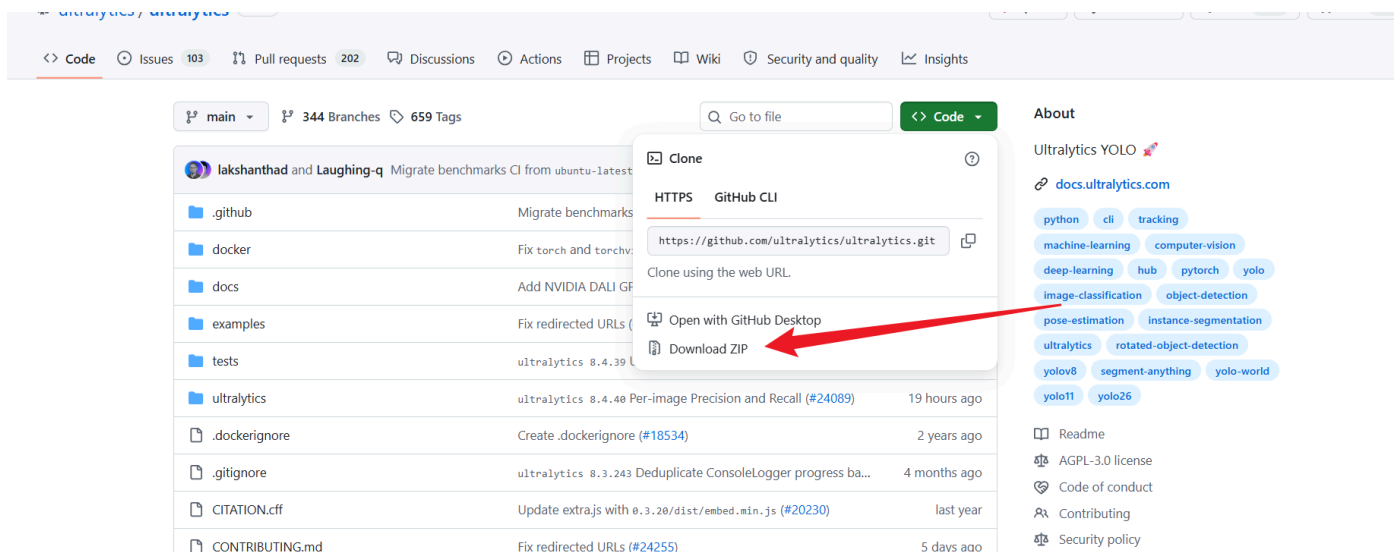
代码块

```
1 pip install ultralytics
```

- 方法二：pip源码安装

<https://github.com/ultralytics/ultralytics>

手动下载和解压



代码块

```
1 # 这一步注意如果使用了魔法上网，需要关闭，不然会有SSL安全问题
2 # (-e参数必须要有，否则后续修改代码无效)
3 pip install -e .
```

- 进行测试

代码块

```
1 cd D:\code\ultralytics-main
2 yolo predict model=yolo26n.pt source=ultralytics/assets/bus.jpg
```

若跑出来下面的效果即可证明成功，注意这里第一次可能需要魔法上网。

```
(yolov26_2) E:\yolo26\ultralytics-main>yolo predict model=yolo26n.pt source=ultralytics/assets/bus.jpg
WARNING Download failure, retrying 1/3 https://github.com/ultralytics/assets/releases/download/v8.4.0/yolo26n.pt... [ASN1: NOT_ENOUGH_DATA] not enough data (_ssl.c:4194)
##### 100.0%
Ultralytics 8.4.40 Python-3.8.20 torch-1.12.1 CUDA:0 (NVIDIA GeForce RTX 3060 Ti, 8192MiB)
E:\yolo26\ultralytics-main\ultralytics\nn\modules\head.py:246: UserWarning: __floordiv__ is deprecated, and its behavior will change in a future version of pytorch. It currently rounds toward 0 (like the 'trunc' function NOT 'floor'). This results in incorrect rounding for negative values. To keep the current behavior, use torch.div(a, b, rounding_mode='trunc'), or for actual floor division, use torch.div(a, b, rounding_mode='floor').
  idx = ori_index[torch.arange(batch_size)[..., None], index // nc] # original index
YOLO26n summary (fused): 122 layers, 2,408,932 parameters, 0 gradients, 5.4 GFLOPs

image 1/1 E:\yolo26\ultralytics-main\ultralytics\assets\bus.jpg: 640x480 4 persons, 1 bus, 13.1ms
Speed: 1.8ms preprocess, 13.1ms inference, 0.4ms postprocess per image at shape (1, 3, 640, 480)
Results saved to C:\lmruns\detect\predict\0m
Learn more at https://docs.ultralytics.com/modes/predict
(yolov26_2) E:\yolo26\ultralytics-main>
```

有概率报错说我们的版本不对

代码块

```
1 pip uninstall pillow -y
2 pip install pillow
```

2. 模型预测分析

2.1 命令预测

yolo task=detect mode=predict model=yolo26n.pt source=ultralytics/assets/bus.jpg

yolo detect predict model=yolo26n.pt source=ultralytics/assets/bus.jpg

task和mode是可以省略的

日志分析：

指令	含义
yolo	调用YOLO的主程序命令
task=detect	指定当前的任务类型为目标检测
mode=predict	指定当前的操作模式为预测(即使用模型对新图片进行推理)
model=yolo26n.pt	指定所使用的模型文件
source=ultralytics/assets/bus.jpg	指定输入数据的来源(要检测的图片)

Ultralytics 8.4.40 Python-3.8.20 torch-1.12.0 CUDA:0 (NVIDIA GeForce RTX 3060 Ti, 8192MiB)

指令	含义

Ultralytics 8.4.40	YOLO版本号
Python-3.8.20	Python版本号
torch-1.12.0	PyTorch 深度学习框架版本
NVIDIA GeForce RTX 3060 Ti, 8192MiB	显卡型号3060Ti,显存8GB

YOLO26n summary (fused): 122 layers, 2,408,932 parameters, 0 gradients, 5.4 GFLOPs

指令	含义
YOLO26n	模型名称
122 layers	神经网络共有122层
2,408,932 parameters	模型约240万个可训练参数(Nano规模，比较轻量)
0 gradients	推理模式下不计算梯度(只有训练模式才有梯度)
5.4 GFLOPs	处理一张图片需要54亿次浮点运算(衡量模型计算复杂度的指标)

image 1/1 E:\yolo26\ultralytics-main\ultralytics\assets\bus.jpg: 640x480 4 persons, 1 bus, 13.1ms

指令	含义
image 1/1	处理第1张图片，总共1张
E:\yolo26\ultralytics-main\ultralytics\assets\bus.jpg	检测的图片路径，图片是bus.jpg
640x480	模型将图片调整到608x640像素进行处理
4 persons, 1 bus	检测到4个人，1辆车
13.1ms	推理耗时13.1ms

Speed: 1.8ms preprocess, 13.1ms inference, 0.4ms postprocess per image at shape (1, 3, 640, 480)

Results saved to [1mruns\detect\predict[0m

指令	含义

1.8ms preprocess	预处理时间（图片读取、缩放、归一化等）
13.1ms inference	图片推理时间（模型计算时间）
0.4ms postprocess	后处理时间（解析检测框、过滤结果等）
shape (1, 3, 640, 480)	输入批次大小和张量形状
Results	结果保存在该路径

处理一张图片的时间总耗时 = 1.8+13.1+0.4 = 15.3ms

2.2 代码测试

predict.py

代码块

```
1  # 图片检测
2  from ultralytics import YOLO
3  yolo = YOLO('yolo26n.pt', task='detect')
4  # result = yolo.predict(source='ultralytics/assets/bus.jpg')
5
6  # 视频检测
7  result = yolo.predict(source='./palace.mp4', show=True)
8
9  # 屏幕检测
10 # result = yolo.predict(source='screen', show=True)
11
12 # 摄像头检测
13 # result = yolo.predict(source=0, show=True)
```

3. 数据集构建

3.1 素材准备

如果是图片可以直接进行标注。
如果是视频需要先转换为图片在进行标注。

视频转图片程序

video_to_img.py

代码块

```
1  import cv2
```

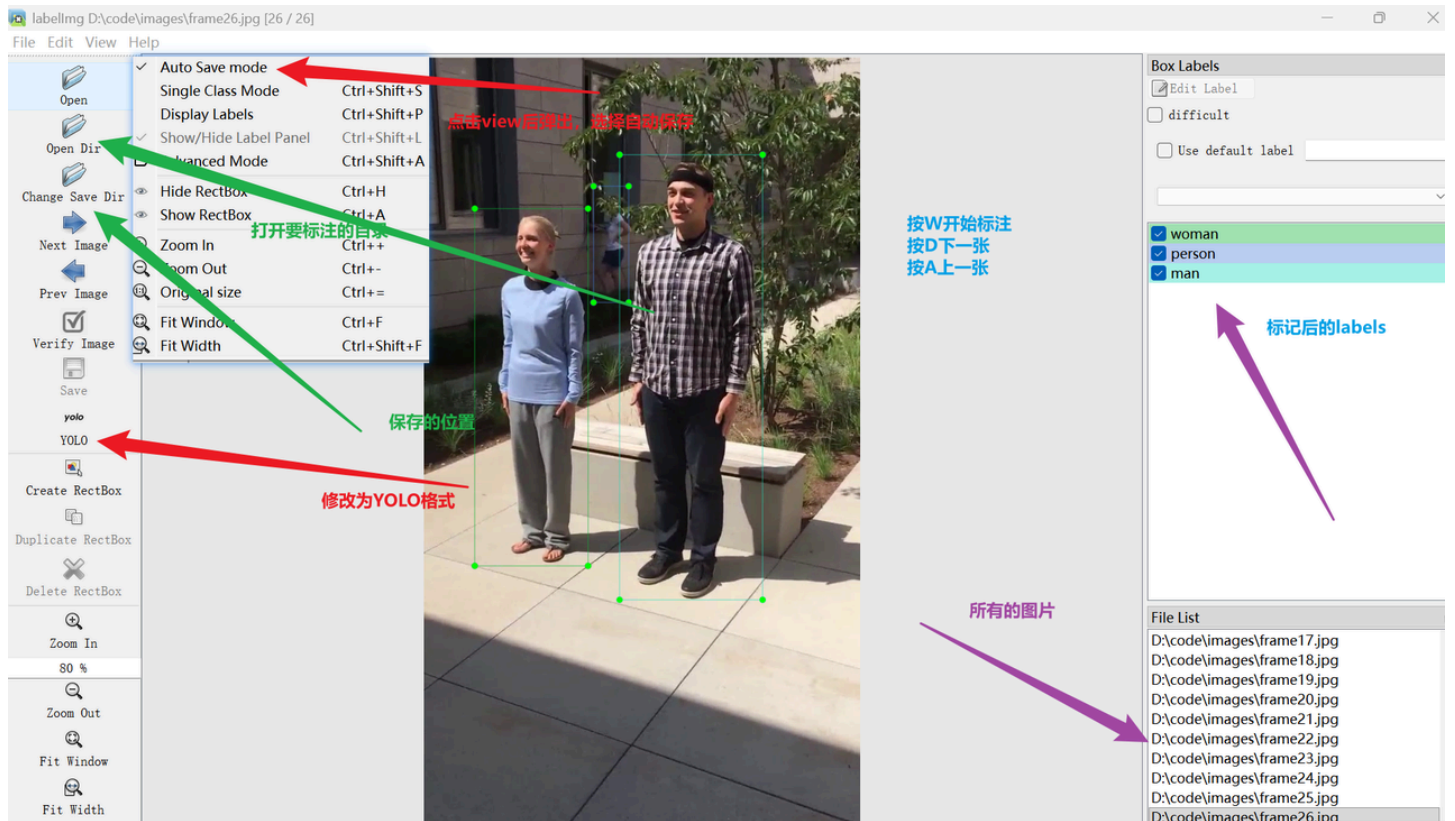
```
2  import os
3
4  os.makedirs("images", exist_ok=True) # 确保目录存在
5  cap = cv2.VideoCapture('palace.mp4')
6
7  count, num, step = 1, 0, 30
8
9  while cap.isOpened():
10     ret, frame = cap.read()
11     if not ret: break
12
13     if num % step == 0:
14         cv2.imwrite(f"images/frame{count}.jpg", frame)
15         count += 1
16     num += 1
17
18  cap.release()
```

3.2 labeling数据集标注

代码块

```
1  # 环境安装
2  pip install labeling
3  # 启动命令
4  labeling
5  # 设置为YOLO格式
```

软件打开后如下所示：



4. 模型训练

4.1 数据准备

数据集准备

训练集和验证集数量是9比1到8比2之间

- images
 - train: 训练集图片
 - val: 验证集图片
- labels
 - train: 训练集标签信息
 - val: 验证集标签信息

准备好数据集以后, 在根目录下创建datasets/helmet_dataset目录, 然后把images和labels都放进去。

准备好配置文件

helmet.yaml

代码块

```
1 path: E:\YOLOV26\ultraalytics-main\datasets\helmet_dataset
2 train: images/train
3 val: images/val
```

```
4
5 nc: 2
6 names: ['helmet', 'no_helmet']
```

4.2 开始训练

可以通过命令行的方式，也可以通过代码方式。

代码块

```
1 yolo task=detect mode=train model=./yolo26n.pt data=helmet.yaml epochs=300
  imgsz=640 batch=-1 workers=4 patience=50 device=0
```

train.py

代码块

```
1 from ultralytics import YOLO
2
3 # 1. 加载模型
4 # 注意：这里加载的是 yolo26n.pt
5 model = YOLO("./yolo26n.pt")
6
7 if __name__ == '__main__':
8     # 2. 开始训练
9     results = model.train(
10         data="helmet.yaml",           # 数据集配置文件路径
11         epochs=300,                   # 建议将训练轮数提升至300，配合早停机制更合理
12         imgsz=640,                   # 明确指定输入图像尺寸为640x640
13
14         batch=-1,                     # 设置为-1开启AutoBatch，自动探测3060Ti能跑
15         workers=4,                   # 设为4以充分利用CPU多线程，避免GPU等待数据
16
17         patience=50,                 # 如果验证集指标连续50轮不提升则提前停止，防止
18         device=0,                     # 明确指定使用第一张显卡进行训练
19
20         task="detect",               # 任务类型：目标检测
21         mode="train",               # 模式：训练
22         verbose=True                 # 显示详细训练日志
23     )
24
25 print("训练完成！")
```


4.3 查看结果

训练完成后，可看到如下结果：

```
E:\YOLOV26\ultralytics-main\ultralytics\nn\modules\head.py:246: UserWarning: __floordiv__ is deprecated, and its behavior will change in a future version of pytorch. It
on NOT 'floor'). This results in incorrect rounding for negative values. To keep the current behavior, use torch.div(a, b, rounding_mode='trunc'), or for actual floor di
.
  idx = ori_index[torch.arange(batch_size)[..., None], index // nc] # original index
    Class      Images  Instances    Box(P)        R    mAP50  mAP50-95): 100% 1/1 13.9it/s 0.1s
      all         20         85      0.81      0.887      0.92      0.584

  Epoch  GPU_mem  box_loss  cls_loss  dfl_loss  Instances    Size
92/300   2.36G    1.06    0.5692   0.003319      2    640: 100% 20/20 11.0it/s 1.8s
E:\YOLOV26\ultralytics-main\ultralytics\nn\modules\head.py:246: UserWarning: __floordiv__ is deprecated, and its behavior will change in a future version of pytorch. It
on NOT 'floor'). This results in incorrect rounding for negative values. To keep the current behavior, use torch.div(a, b, rounding_mode='trunc'), or for actual floor di
.
  idx = ori_index[torch.arange(batch_size)[..., None], index // nc] # original index
    Class      Images  Instances    Box(P)        R    mAP50  mAP50-95): 100% 1/1 13.2it/s 0.1s
      all         20         85      0.904      0.809      0.931      0.591
EarlyStopping: Training stopped early as no improvement observed in last 50 epochs. Best results observed at epoch 42, best model saved as best.pt.
To update EarlyStopping(patience=50) pass a new patience value, i.e. 'patience=300' or use 'patience=0' to disable EarlyStopping.

92 epochs completed in 0.056 hours.
Optimizer stripped from E:\YOLOV26\ultralytics-main\runs\detect\train-3\weights\last.pt, 5.4MB
Optimizer stripped from E:\YOLOV26\ultralytics-main\runs\detect\train-3\weights\best.pt, 5.4MB

Validating E:\YOLOV26\ultralytics-main\runs\detect\train-3\weights\best.pt...
Ultralytics 8.4.40 Python-3.8.20 torch-1.12.1 CUDA:0 (NVIDIA GeForce RTX 3060 Ti, 8192MiB)
YOLO26n summary (fused): 122 layers, 2,375,226 parameters, 0 gradients, 5.2 GFLOPs
    Class      Images  Instances    Box(P)        R    mAP50  mAP50-95): 100% 1/1 9.9it/s 0.1s
      all         20         85      0.781      0.8      0.902      0.617
    helmet      10         14      0.736      0.929      0.963      0.722
    no_helmet    16         71      0.827      0.672      0.84      0.513
Speed: 0.3ms preprocess, 1.3ms inference, 0.0ms loss, 0.1ms postprocess per image
Results saved to E:\YOLOV26\ultralytics-main\runs\detect\train-3
训练完成!
```

文件名	说明
BoxF1_curve.png	F1曲线:展示了不同置信度阈值下F1分数的变化趋势。F1是精确率和召回率的调和均数，峰值越高说明模型越好。
BoxP_curve.png	精确率曲线:展示了不同置信度阈值下精确率的变化。精确率=TP/(TP+FP)，反映模型预测为正例的样本中实际为正例的比例。
BoxPR_curve.png	F1曲线:展示了不同置信度阈值下F1分数的变化趋势。F1是精确率和召回率的调和平均数，峰值越高说明模型越好。
BoxP_curve.png	PR曲线(精确率-召回率曲线):综合展示了不同置信度阈值下精确率和召回率的关系。曲线下面积(AP)越大，模型性能越好。
BoxR_curve.png	召回率曲线:展示了不同置信度阈值下召回率的变化。召回率=TP/(TP+FN)，反映模型找出正样本的能力。
confusion_matrix.png	混淆矩阵:展示每个类别的真实标签与预测标签的对应关系。对角线越亮，分类效果越好。
confusion_matrix_normalized.png	归一化混淆矩阵:将混淆矩阵按行归一化(百分比形式)，更直观地看出每个类别的正确率与错误分类情况
labels.jpg	训练集标签分布图:展示所有训练样本中标注框的分布情况，包括标注框的中心点分布大小分布、类别数量等。

train_batch0.jpg train_batch1.jpg train_batch2.jpg	训练批次样本图:展示训练过程中前几个批次的图像及其标注框，帮助检查数据加载是否正确、标注是否对齐。
train_batch40.jpg train_batch41.jpg train_batch42.jpg	训练后期批次样本图:训练结束前最后几个批次的图像，用于对比训练过程中的数据变化。
val_batch0_labels.jpg	验证集标签图:展示验证集批次图像的真实标注框。
val_batch0_pred.jpg	验证集预测图:展示同一验证集批次的模型预测结果(标注框+置信度)，用于直观对比预测与真实标签的差异。
results.csv	训练日志CSV文件:包含每一轮的详细指标数据(如损失值、精确率、召回率、mAP等可用于自定义绘图或分析)。
results.png	训练结果汇总图:将多个关键指标(如损失、精确率、召回率、mAP)随训练轮次的变化趋势绘制在一张图中，便于快速评估训练效果
weights/best.pt weights/last.pt	最好的模型 最后一轮的模型

测试一下我们的模型：

代码块

```
1 yolo detect predict model=runs\detect\train-3\weights\best.pt
  source=datasets\helmet_dataset\images\val\99.jpg
```



5. 导出ONNX模型

5.1 源码修改1

head.py

代码块

```

1      def forward(
2          self, x: list[torch.Tensor]
3      ) -> dict[str, torch.Tensor] | torch.Tensor | tuple[torch.Tensor,
4          dict[str, torch.Tensor]]:
5          """Concatenates and returns predicted bounding boxes and class
6              probabilities."""
7          # *****onnx导出增加代码*****
8          y = []
9          if self.end2end:
10             # 针对端到端导出模式，使用特定的卷积层 (one2one_cv2/cv3)
11             for i in range(self.nl):
12                 t1 = self.one2one_cv2[i](x[i])
13                 t2 = self.one2one_cv3[i](x[i])
14                 y.append(t1)
15                 y.append(t2)
16             return y

```

```

15         else:
16             # 标准训练/推理模式, 使用常规卷积层 (cv2/cv3)
17             for i in range(self.nl):
18                 t1 = self.cv2[i](x[i])
19                 t2 = self.cv3[i](x[i])
20                 y.append(t1)
21                 y.append(t2)
22             return y
23         # *****onnx导出结束*****
24         preds = self.forward_head(x, **self.one2many)
25         if self.end2end:
26             x_detach = [xi.detach() for xi in x]
27             one2one = self.forward_head(x_detach, **self.one2one)
28             preds = {"one2many": preds, "one2one": one2one}
29         if self.training:
30             return preds
31         y = self._inference(preds["one2one"] if self.end2end else preds)
32         if self.end2end:
33             y = self.postprocess(y.permute(0, 2, 1))
34         return y if self.export else (y, preds)

```

5.2 源码修改2

model.py

代码块

```

1     def _load(self, weights: str, task=None) -> None:
2         """Load a model from a checkpoint file or initialize it from a weights
3         file.
4
5         This method handles loading models from either .pt checkpoint files or
6         other weight file formats. It sets up the
7         model, task, and related attributes based on the loaded weights.
8
9         Args:
10             weights (str): Path to the model weights file to be loaded.
11             task (str, optional): The task associated with the model. If None,
12             it will be inferred from the model.
13
14         Raises:
15             FileNotFoundError: If the specified weights file does not exist or
16             is inaccessible.
17             ValueError: If the weights file format is unsupported or invalid.
18
19         Examples:

```

```

16         >>> model = Model()
17         >>> model._load("yolo26n.pt")
18         >>> model._load("path/to/weights.pth", task="detect")
19         """
20         if weights.lower().startswith(("https://", "http://", "rtsp://",
"rtmp://", "tcp://", "ul://")):
21             weights = checks.check_file(weights,
download_dir=SETTINGS["weights_dir"]) # download and return local file
22             weights = checks.check_model_file_from_stem(weights) # add suffix,
i.e. yolo26 -> yolo26n.pt
23
24             if str(weights).rpartition(".")[1] == "pt":
25                 self.model, self.ckpt = load_checkpoint(weights)
26                 self.task = self.model.task
27                 self.overrides = self.model.args =
self._reset_ckpt_args(self.model.args)
28                 self.ckpt_path = self.model.pt_path
29             else:
30                 weights = checks.check_file(weights) # runs in all cases, not
redundant with above call
31                 self.model, self.ckpt = weights, None
32                 self.task = task or guess_model_task(weights)
33                 self.ckpt_path = weights
34                 self.overrides["model"] = weights
35                 self.overrides["task"] = self.task
36                 self.model_name = weights
37
38                 print("===== onnx start =====")
39                 import torch
40                 dummy_input = torch.randn(1, 3, 640, 640)
41                 input_names = ["data"]
42                 output_names = ["reg1", "cls1", "reg2", "cls2", "reg3", "cls3"]
43                 torch.onnx.export(
44                     self.model,
45                     dummy_input,
46                     "weights/yolo26n-float.onnx",
47                     verbose=False,
48                     opset_version=11,
49                     do_constant_folding=True,
50                     input_names=input_names,
51                     output_names=output_names,
52                 )
53                 print("===== onnx end =====")

```

5.3 创建导出文件导出

export.py

代码块

```
1  from ultralytics import YOLO
2
3  model = YOLO("best.pt" )
4
5  model.export(format="onnx")
```

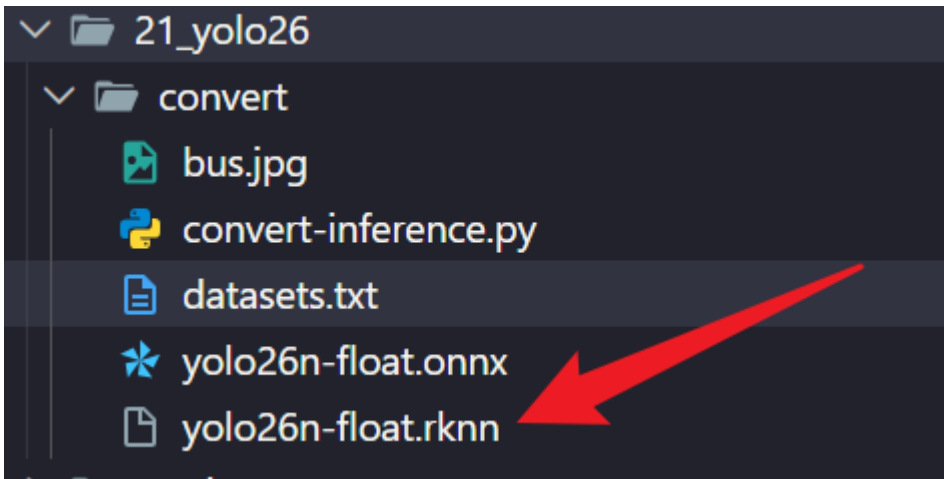
执行后即可导出onnx模型。

6. 转换RKNN模型

通过 RKNN-Toolkit 将 ONNX 模型量化为 RKNN 格式（生成 `yolo26n.rknn`）。

创建一个convert目录。执行 `python convert-inference.py`

执行完成后即可看到，已经转换为了rknn模型。



代码块

```
1  import os
2  import urllib
3  import traceback
4  import time
5  import sys
6  import numpy as np
7  import cv2
8  from rknn.api import RKNN
9  from math import exp
10
11  ONNX_MODEL = './yolo26n-float.onnx'
12  RKNN_MODEL = './yolo26n-float.rknn'
13  DATASET = './datasets.txt'
14
```



```

15  QUANTIZE_ON = True
16
17  # CLASSES = ['person', 'bicycle', 'car', 'motorcycle', 'airplane', 'bus',
18  #            'train', 'truck', 'boat', 'traffic light',
19  #            'fire hydrant', 'stop sign', 'parking meter', 'bench', 'bird',
20  #            'cat', 'dog', 'horse', 'sheep', 'cow',
21  #            'elephant', 'bear', 'zebra', 'giraffe', 'backpack', 'umbrella',
22  #            'handbag', 'tie', 'suitcase', 'frisbee',
23  #            'skis', 'snowboard', 'sports ball', 'kite', 'baseball bat',
24  #            'baseball glove', 'skateboard', 'surfboard',
25  #            'tennis racket', 'bottle', 'wine glass', 'cup', 'fork', 'knife',
26  #            'spoon', 'bowl', 'banana', 'apple',
27  #            'sandwich', 'orange', 'broccoli', 'carrot', 'hot dog', 'pizza',
28  #            'donut', 'cake', 'chair', 'couch',
29  #            'potted plant', 'bed', 'dining table', 'toilet', 'tv', 'laptop',
30  #            'mouse', 'remote', 'keyboard', 'cell phone',
31  #            'microwave', 'oven', 'toaster', 'sink', 'refrigerator', 'book',
32  #            'clock', 'vase', 'scissors', 'teddy bear',
33  #            'hair drier', 'toothbrush']
34
35  CLASSES = ['helmet', 'no_helmet']
36
37  meshgrid = []
38
39  class_num = len(CLASSES)
40  headNum = 3
41  strides = [8, 16, 32]
42  mapSize = [[80, 80], [40, 40], [20, 20]]
43  nmsThresh = 0.4
44  objectThresh = 0.25
45
46  input_imgH = 640
47  input_imgW = 640
48
49  class DetectBox:
50      def __init__(self, classId, score, xmin, ymin, xmax, ymax):
51          self.classId = classId
52          self.score = score
53          self.xmin = xmin
54          self.ymin = ymin
55          self.xmax = xmax
56          self.ymax = ymax
57
58  def GenerateMeshgrid():
59      for index in range(headNum):
60          for i in range(mapSize[index][0]):
61              for j in range(mapSize[index][1]):

```

```
54         meshgrid.append(j + 0.5)
55         meshgrid.append(i + 0.5)
56
57 def IOU(xmin1, ymin1, xmax1, ymax1, xmin2, ymin2, xmax2, ymax2):
58     xmin = max(xmin1, xmin2)
59     ymin = max(ymin1, ymin2)
60     xmax = min(xmax1, xmax2)
61     ymax = min(ymax1, ymax2)
62
63     innerWidth = xmax - xmin
64     innerHeight = ymax - ymin
65
66     innerWidth = innerWidth if innerWidth > 0 else 0
67     innerHeight = innerHeight if innerHeight > 0 else 0
68
69     innerArea = innerWidth * innerHeight
70
71     area1 = (xmax1 - xmin1) * (ymax1 - ymin1)
72     area2 = (xmax2 - xmin2) * (ymax2 - ymin2)
73
74     total = area1 + area2 - innerArea
75
76     return innerArea / total
77
78 def handleResult(detectResult):
79     predBoxs = []
80
81     sort_detectboxs = sorted(detectResult, key=lambda x: x.score, reverse=True)
82     for i in range(len(sort_detectboxs)):
83         xmin1 = sort_detectboxs[i].xmin
84         ymin1 = sort_detectboxs[i].ymin
85         xmax1 = sort_detectboxs[i].xmax
86         ymax1 = sort_detectboxs[i].ymax
87         classId = sort_detectboxs[i].classId
88         predBoxs.append(sort_detectboxs[i])
89     return predBoxs
90
91 def sigmoid(x):
92     return 1 / (1 + exp(-x))
93
94 def postprocess(out, img_h, img_w):
95     print('postprocess ... ')
96
97     detectResult = []
98     output = []
99     for i in range(len(out)):
100         print(out[i].shape)
```



```

101         output.append(out[i].reshape((-1)))
102
103     scale_h = img_h / input_imgH
104     scale_w = img_w / input_imgW
105
106     gridIndex = -2
107     cls_index = 0
108     cls_max = 0
109
110     for index in range(headNum):
111         reg = output[index * 2 + 0]
112         cls = output[index * 2 + 1]
113
114         for h in range(mapSize[index][0]):
115             for w in range(mapSize[index][1]):
116                 gridIndex += 2
117
118                 if 1 == class_num:
119                     cls_max = sigmoid(cls[0 * mapSize[index][0] *
mapSize[index][1] + h * mapSize[index][1] + w])
120                     cls_index = 0
121                 else:
122                     for cl in range(class_num):
123                         cls_val = cls[cl * mapSize[index][0] * mapSize[index]
[1] + h * mapSize[index][1] + w]
124                         if 0 == cl:
125                             cls_max = cls_val
126                             cls_index = cl
127                         else:
128                             if cls_val > cls_max:
129                                 cls_max = cls_val
130                                 cls_index = cl
131                     cls_max = sigmoid(cls_max)
132
133                 if cls_max > objectThresh:
134
135                     x1 = (meshgrid[gridIndex + 0] - reg[0*mapSize[index][0] *
mapSize[index][1] + h * mapSize[index][1] + w]) * strides[index]
136                     y1 = (meshgrid[gridIndex + 1] - reg[1*mapSize[index][0] *
mapSize[index][1] + h * mapSize[index][1] + w]) * strides[index]
137                     x2 = (meshgrid[gridIndex + 0] + reg[2*mapSize[index][0] *
mapSize[index][1] + h * mapSize[index][1] + w]) * strides[index]
138                     y2 = (meshgrid[gridIndex + 1] + reg[3*mapSize[index][0] *
mapSize[index][1] + h * mapSize[index][1] + w]) * strides[index]
139
140                     print(index)
141                     print('x1,y1,x2,y2:', x1, y1, x2, y2)

```

```

142
143
144         xmin = x1 * scale_w
145         ymin = y1 * scale_h
146         xmax = x2 * scale_w
147         ymax = y2 * scale_h
148
149         xmin = xmin if xmin > 0 else 0
150         ymin = ymin if ymin > 0 else 0
151         xmax = xmax if xmax < img_w else img_w
152         ymax = ymax if ymax < img_h else img_h
153
154         box = DetectBox(cls_index, cls_max, xmin, ymin, xmax, ymax)
155         detectResult.append(box)
156
157     # NMS
158     print('detectResult:', len(detectResult))
159     predBox = handleResult(detectResult)
160
161     return predBox
162
163 def export_rknn_inference(img):
164     # Create RKNN object
165     rknn = RKNN(verbose=False)
166
167     # pre-process config
168     print('--> Config model')
169     rknn.config(mean_values=[[0,0,0]], std_values=[[255,255,255]],
170 quantized_algorithm='normal', quantized_method='channel',
171 target_platform='rk3588')
172     print('done')
173
174     # Load ONNX model
175     print('--> Loading model')
176     # ret = rknn.load_onnx(model=ONNX_MODEL, outputs=[ 'reg1', 'cls1', 'reg2',
177 'cls2', 'reg3', 'cls3'])
178     ret = rknn.load_onnx(model=ONNX_MODEL)
179     if ret != 0:
180         print('Load model failed!')
181         exit(ret)
182     print('done')
183
184     # Build model
185     print('--> Building model')
186     ret = rknn.build(do_quantization=QUANTIZE_ON, dataset=DATASET,
187 rknn_batch_size=1)
188     if ret != 0:

```

```
185         print('Build model failed!')
186         exit(ret)
187     print('done')
188
189     # Export RKNN model
190     print('--> Export rknn model')
191     ret = rknn.export_rknn(RKNN_MODEL)
192     if ret != 0:
193         print('Export rknn model failed!')
194         exit(ret)
195     print('done')
196
197     # Init runtime environment
198     print('--> Init runtime environment')
199     ret = rknn.init_runtime()
200     # ret = rknn.init_runtime(target='rk3566')
201     if ret != 0:
202         print('Init runtime environment failed!')
203         exit(ret)
204     print('done')
205
206     # Inference
207     print('--> Running model')
208     outputs = rknn.inference(inputs=[img])
209     rknn.release()
210     print('done')
211
212     return outputs
213
214 if __name__ == '__main__':
215     print('This is main ...')
216     GenerateMeshgrid()
217
218     img_path = '16.jpg'
219     orig_img = cv2.imread(img_path)
220     img_h, img_w = orig_img.shape[:2]
221
222
223     origimg = cv2.resize(orig_img, (input_imgW, input_imgH),
224                          interpolation=cv2.INTER_LINEAR)
225     origimg = cv2.cvtColor(origimg, cv2.COLOR_BGR2RGB)
226
227     img = np.expand_dims(origimg, 0)
228
229     outputs = export_rknn_inference(img)
230
231     out = []
```

```

231     for i in range(len(outputs)):
232         out.append(outputs[i])
233
234     predbox = postprocess(out, img_h, img_w)
235
236     print(len(predbox))
237
238     for i in range(len(predbox)):
239         xmin = int(predbox[i].xmin)
240         ymin = int(predbox[i].ymin)
241         xmax = int(predbox[i].xmax)
242         ymax = int(predbox[i].ymax)
243         classId = predbox[i].classId
244         score = predbox[i].score
245
246         cv2.rectangle(orig_img, (xmin, ymin), (xmax, ymax), (0, 255, 0), 2)
247         ptext = (xmin, ymin)
248         title = CLASSES[classId] + ":%.2f" % (score)
249         cv2.putText(orig_img, title, ptext, cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0,
0, 255), 2, cv2.LINE_AA)
250
251     cv2.imwrite('./test_rknn_result.jpg', orig_img)
252

```

7. C++ 推理部署

yolov26的代码结构什么的与前面的yolov5都是类似的，在这里我就不一一贴出来了，直接看课程的讲解就行。

注意点：

postprocess.cpp 文件 类别修改为2类：static int class_num = 2;

yolo26.cpp文件 static std::vector<std::string> g_classes = {"helmet", "no_helmet"};

